KISS

Keep It Simple Principle

Prefer the simplest thing that works

DEFINITION

Favor simple, obvious solutions; complexity must earn its place.

"Simple" means the fewest moving parts a reader must hold in their head to be sure the code is correct — not the fewest characters. Terse, clever one-liners are often the opposite of simple. KISS targets accidental complexity, the kind we add ourselves (needless layers, premature generality), while accepting the essential complexity the problem genuinely has.

WHY IT MATTERS

Code is read and changed far more than it is written, so the cost that matters is the next person's comprehension — often you, months later. Every clever trick, extra layer and speculative hook is a tax paid on every future change, usually to buy flexibility that never gets used. Simpler code has fewer places to hide bugs and a shorter path from question to answer.

Vasyl Krupka · Ukraine

How it works

Essential complexity	Complexity inherent to the problem — it must be modeled and cannot be wished away. KISS does not fight this.
Accidental complexity	Complexity we introduce: clever tricks, extra indirection, speculative abstraction. This is what KISS removes.
Reader	The next maintainer who must understand the code to change it safely. The audience you optimize for, not the compiler.

HOW TO APPLY

1. Write the most obvious version that works; make it clever only if a measurement says you must.
2. Count the concepts a reader must learn to follow the code — fewer is the goal.
3. Delete speculative flexibility (config, hooks, layers) that no current requirement uses.
4. Prefer a language or stdlib feature over hand-rolled machinery for the same result.

LITMUS TEST

Could a new teammate follow this in one pass? Is each abstraction paying for itself with a concrete, present need? If not, simplify.

Vasyl Krupka · Ukraine

Smell → fix

Bit-twiddling and puzzle operators make the reader stop and decode:

```
function isEven(n) { return !(n & 1) } // bitwise trick
const found = !!~list.indexOf(x) // !!~ puzzle
```

↓ CHOOSE THE OBVIOUS SOLUTION

```
function isEven(n) { return n % 2 === 0 } // says what it means
const found = list.includes(x) // intent is obvious
```

Same behavior, but each line states its intent. Keep the clever version only for a proven hot path, with a comment explaining why it earns its keep.

USE WHEN

- ✓ Choosing between a clever and an obvious solution
- ✓ Reviewing over-engineered code

AVOID WHEN

- ▲ Over-simplifying past real requirements

— Trade-offs & pitfalls

PROS

+ Readable, maintainable
+ Faster onboarding

CONS

− "Simple" is subjective
− Can hide flexibility you'll need

COMMON PITFALLS

▲ Mistaking "short" for "simple" — dense one-liners and bit-tricks add cognitive load, not clarity.
▲ Over-simplifying past the requirements, then bolting complexity back on awkwardly later.
▲ Cargo-culting a pattern or framework for a problem a plain function would solve.

WHERE YOU SEE IT

- A plain function instead of a Strategy class hierarchy when there is only one algorithm.
- list.includes(x) instead of a clever !!~indexOf trick.
- One well-named module instead of a four-layer abstraction for a simple CRUD endpoint.

SEE ALSO

YAGNI	don't build for needs you don't have yet — the main source of complexity
DRY	one clear source is simpler than scattered copies
SoC	separating concerns keeps each part individually simple
CoI	small composed parts beat deep inheritance trees

— Interview & sources

Is KISS at odds with design patterns?

Patterns add structure and indirection. They earn their place when complexity is real; applied preemptively they violate KISS and YAGNI.

"Simple" is subjective — how do you make it concrete?

Proxy it: fewer concepts to learn, fewer branches, shorter call chains, readable in one pass. Optimize for the next reader, not the compiler.

Where's the limit of KISS?

Do not oversimplify past real requirements. Essential complexity must be modeled; KISS targets the accidental kind we add ourselves.

KISS vs DRY — can they clash?

Yes. Aggressive DRY can add indirection that hurts clarity. When a small duplication reads simpler than a shared abstraction, KISS wins — prefer the obvious code.

REMEMBER

Reach for the simplest thing that fully solves the problem — complexity must earn its place.

SOURCES

Often attributed to Kelly Johnson (Lockheed Skunk Works): "keep it simple, stupid"

Brian Kernighan, P.J. Plauger — "The Elements of Programming Style" (1978): write clearly, debugging is harder than writing